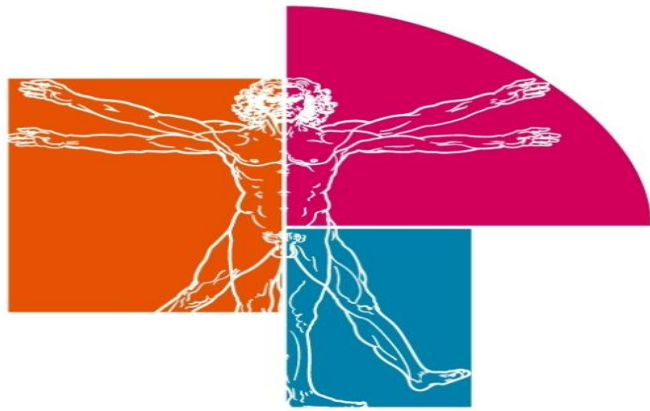




IES LeonardoDaVinci  
ALBACETE



# TEMA 3

## DESARROLLO DE APLICACIONES WEB CON JAVA



# OBJETIVOS

- Entender la importancia de guardar el estado del cliente en aplicaciones Web
- Realizar aplicaciones Web Java que utilicen cookies.
- Realizar aplicaciones Web Java que utilicen sesiones.



# INDICE

1. Guardar el estado del cliente
2. Cookies
  - 2.1. Crear una cookie
  - 2.2. Dar un tiempo de vida a una Cookie
  - 2.3. Enviar una Cookie
  - 2.4. Recuperar Cookies
  - 2.5. Obtener el valor de una Cookie



### 3. Seguimiento de sesión

3.1. Obtener una sesión

3.2. Almacenar y obtener datos desde la sesión

3.3. Invalidar una sesión

3.4. Comunicación entre Servlets

3.5. Ceder el control desde un servlet a otro o una JSP

3.6. Sesiones en JSP



# 1. Guardar el estado del cliente

- HTTP es un protocolo sin estado. Cada conexión asociada a peticiones y a respuestas es independiente de las demás.
- Entre una petición y otra (del mismo usuario), el servidor HTTP olvida la petición previa. Por eso el contenedor Web debe crear un mecanismo para almacenar la información de sesión de un mismo usuario.
- El API Servlet proporciona dos formas de seguir la pista al estado de un cliente: **cookies y/o sesiones**.



## Cookies

- Las Cookies son un mecanismo que el servlet utiliza para mantener en el cliente una pequeña cantidad de información asociada con el usuario.
- En las cookies se puede guardar información del usuario como contraseñas, preferencias, enlaces visitados, etc.

## Seguimiento de Sesión

- El seguimiento de sesión es un mecanismo que los servlets utilizan para conservar el estado de un mismo cliente sobre la serie de peticiones durante algún periodo de tiempo.



## 2. Cookies

- Las **Cookies** son un mecanismo mediante el cual un **servidor** (o un servlet, como parte de un servidor) **envía información al cliente para ser almacenada**. Posteriormente el servidor puede recuperar esos datos en la siguiente conexión del cliente.
- Los servlet envían cookies al cliente añadiendo campos a las cabeceras de respuesta HTTP.
- Los clientes devuelven las cookies automáticamente añadiendo campos a las cabeceras de peticiones HTTP.



- Aspectos fundamentales de las cookies:
  - ✓ Son pequeños ficheros de texto.
  - ✓ Las envía el servidor Web.
  - ✓ Se almacenan en el ordenador del cliente.
  - ✓ El navegador las devuelve cuando se visita el mismo sitio Web o dominio.
  - ✓ Todas las cookies asociadas a un dominio son enviadas en cada petición al servidor web.
  - ✓ Las cookies tienen un tiempo de vida limitado y son eliminadas por el navegador del cliente al final de ese proceso.



## ○ Ventajas:

- ✓ Identificar a un usuario durante una sesión de comercio electrónico.
- ✓ Evitar escribir el nombre de usuario y la password cada vez que se visita un determinado dominio.
- ✓ Personalizar un dominio, colores, elementos.
- ✓ Publicidad enfocada.
- ✓ No son un serio problema de seguridad.
- ✓ Las cookies nunca son interpretadas o ejecutadas de ninguna forma, y no pueden usarse para insertar virus o atacar nuestro sistema.



## ○ Desventajas:

- ✓ Aunque no presentan un serio problema de seguridad, si que presentan un significativo **problema de privacidad**.
- ✓ Debido a estos problemas reales de privacidad, **algunos usuarios desactivan las cookies**.
- ✓ Por eso, puede ser un **problema que nuestra web dependa de las cookies**.
- ✓ Como programadores de servlets que podemos usar cookies, **no deberíamos confiar a las cookies información particularmente sensible**, ya que el usuario podría correr el riesgo de que alguien accediera a su ordenador o a sus ficheros de cookies.



- **Para enviar una cookie:**
  - ✓ Crear un objeto Cookie.
  - ✓ Seleccionar cualquier atributo.
  - ✓ Enviar la cookie
- **Para obtener información de una cookie:**
  - ✓ Recuperar todas las cookies de la petición del usuario.
  - ✓ Buscar la cookie o cookies con el nombre que te interesa, utilizando las técnicas de programación estándar.
  - ✓ Obtener los valores de las cookies que hayas encontrado.



- Con el API de la **clase Cookie** tendremos toda la información y posibilidades de operación sobre las cookies en Java.

## Constructor Summary

**Cookie**(java.lang.String name, java.lang.String value) Constructs a cookie with a specified name and value.

## Method Summary

|                  |                                                                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| java.lang.Object | <b><u>clone()</u></b><br>Overrides the standard java.lang.Object.clone method to return a copy of this cookie.                                                      |
| java.lang.String | <b><u>getComment()</u></b><br>Returns the comment describing the purpose of this cookie, or null if the cookie has no comment.                                      |
| java.lang.String | <b><u>getDomain()</u></b> Returns the domain name set for this cookie.                                                                                              |
| int              | <b><u>getMaxAge()</u></b><br>Returns the maximum age of the cookie, specified in seconds, By default, -1 indicating the cookie will persist until browser shutdown. |
| java.lang.String | <b><u>getName()</u></b> Returns the name of the cookie.                                                                                                             |



|                  |                                                                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| java.lang.String | <u><b>getPath()</b></u> Returns the path on the server to which the browser returns this cookie.                                                                      |
| boolean          | <u><b>getSecure()</b></u><br>Returns true if the browser is sending cookies only over a secure protocol, or false if the browser can send cookies using any protocol. |
| java.lang.String | <u><b>getValue()</b></u> Returns the value of the cookie.                                                                                                             |
| int              | <u><b>getVersion()</b></u> Returns the version of the protocol this cookie complies with.                                                                             |
| void             | <u><b>setComment(java.lang.String purpose)</b></u><br>Specifies a comment that describes a cookie's purpose.                                                          |
| void             | <u><b>setDomain(java.lang.String pattern)</b></u><br>Specifies the domain within which this cookie should be presented.                                               |



|      |                                                                                                                                                          |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| void | <u><b>setMaxAge</b></u> (int expiry) Sets the maximum age of the cookie in seconds.                                                                      |
| void | <u><b>setPath</b></u> (java.lang.String uri)<br>Specifies a path for the cookie to which the client should return the cookie.                            |
| void | <u><b>setSecure</b></u> (boolean flag)<br>Indicates to the browser whether the cookie should only be sent using a secure protocol, such as HTTPS or SSL. |
| void | <u><b>setValue</b></u> (java.lang.String newValue)<br>Assigns a new value to a cookie after the cookie is created.                                       |
| void | <u><b>setVersion</b></u> (int v) Sets the version of the cookie protocol this cookie complies with.                                                      |



## 2.1. Crear una Cookie

- El **constructor por defecto** de la clase `javax.servlet.http.Cookie` **crea una cookie con un nombre inicial y un valor** (que siempre será un objeto). Se puede cambiar el valor posteriormente utilizando el método `setValue`.
- Lo más lógico es utilizar el constructor:  
`Cookie(String name, String value)`
- Ejemplo:  
`Cookie cok=new Cookie("user",usuario);`



## 2.2. Dar un tiempo de vida a una Cookie

- Podemos establecer un tiempo de vida (en segundos) a la cookie con el método `setMaxAge(int expiry)`.
- Ejemplo: `nombreCookie.setMaxAge(60*3);`

## 2.3. Enviar una Cookie

- Las cookies se envían como cabeceras en la respuesta al cliente, se añaden con el método `addCookie` de la clase `HttpServletResponse`.
- Ejemplo: `response.addCookie(cok);`



## 2.4. Recuperar Cookies

- Los clientes devuelven las cookies como campos añadidos a las cabeceras de petición HTTP.
- Para recuperar una cookie, **debemos recuperar todas las cookies utilizando el método `getCookies` de la clase `HttpServletRequest`.**
- El método `getCookies` **devuelve un array de objetos `Cookie`**, en el que podemos buscar la cookie o cookies que queramos.
- Para **obtener el nombre de una cookie**, utilizamos el método `getName`.



- Ejemplo:

```
Cookie [] misCookies=request.getCookies();
```

- La primera posición del array de Cookies la ocupa una que se crea por defecto al entrar en el servidor, es una identificación de sesión realmente (se verá después) y tiene una descripción parecida a:

```
misCookies[0]: JSESSIONID 8BCBFB00A604975D20B703D76B9CE251
```



## 2.5. Obtener el valor de una Cookie

- Para obtener el valor de una cookie, se utiliza el **método getValue**.

- Ejemplo:

```
for(int i=0;i<misCookies.length;i++) {  
  
    Cookie unaCookie=misCookies[i];  
  
    if(unaCookie.getName().equals(nombreC)) {  
  
        valorC=unaCookie.getValue();  
  
    }  
  
}
```



# Ejemplo completo con Cookie

- Página de inicio

```
<HTML>
<HEAD>
  <META HTTP-EQUIV="Content-Type" CONTENT="text/html; charset=windows-1252"/>
  <TITLE>Creando Cookies</TITLE>
</HEAD>
<BODY>
  <H2>Introduce nombre y te crearé una cookie.</H2>
  <form action="HazCookie">
    Nombre : <input type="text" name="nombre"
      size="15" maxlength="15">
    <input type="submit" value="Crear Cookie">
  </form>
</BODY>
</HTML>
```



## ○ Servlet en donde creo la Cookie

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.PrintWriter;
import java.io.IOException;

public class HazCookie extends HttpServlet {
    private static final String CONTENT_TYPE = "text/html; charset=windows-1252";
    public void init(ServletConfig config) throws ServletException{
        super.init(config);
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
                                                                    ServletException, IOException{
        response.setContentType(CONTENT_TYPE);
        PrintWriter out = response.getWriter();
    }
}
```



```
String usuario=request.getParameter("nombre");  
Cookie cok=new Cookie("user",usuario); //necesita un objeto para cada parámetro  
cok.setMaxAge(60*3);  
response.addCookie(cok);
```

```
out.println("<html>");  
out.println("<head><title>HazCookie</title></head>");  
out.println("<body>");  
out.println("<p>He creado una cookie con el nombre : "+usuario+"</p>");  
out.println("</body></html>");  
out.close();  
}  
}
```



## ○ Servlet en donde recupero y muestro la Cookie

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.PrintWriter;
import java.io.IOException;

public class LeerCookie extends HttpServlet
{
    private static final String CONTENT_TYPE = "text/html; charset=windows-1252";

    public void init(ServletConfig config) throws ServletException
    {
        super.init(config);
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException

    {
        response.setContentType(CONTENT_TYPE);
        PrintWriter out = response.getWriter();
    }
}
```



```
Cookie [ ] misCookies=request.getCookies();
String nombreC="user", valorC=null;
for(int i=0;i<misCookies.length;i++)
{
    Cookie unaCookie=misCookies[i];
    if(unaCookie.getName().equals(nombreC))
    {
        valorC=unaCookie.getValue();
    }
}

out.println("<html>");
out.println("<head><title>LeerCookie</title></head>");
out.println("<body>");
if(valorC!=null){
out.println("<p>Hola "+valorC+"</p>");
}else
{
    out.println("<p>Hola usuario desconocido/p>");
}
out.println("</body></html>");
out.close();
}
```



### 3. Seguimiento de Sesión

- El seguimiento de sesión es otro mecanismo que los servlets utilizan para **mantener el estado sobre la serie de peticiones desde un mismo usuario** (peticiones originadas desde el mismo navegador) durante un periodo de tiempo.
- **Aspectos fundamentales** de las sesiones:
  - ✓ La **información se almacena en el servidor**.
  - ✓ Se utiliza la **interfaz HttpSession**, de la API de Servlets.
  - ✓ El uso de sesiones trae asociada una búsqueda en el servidor dependiendo de la petición del cliente.



- ✓ Internamente usa cookies para relacionar cliente con espacio de memoria, si es posible; si no puede utilizar otros métodos (como reescritura de URL).
- ✓ Usar sesiones en servlets es bastante sencillo.
- ✓ Las sesiones son compartidas por los servlets a los que accede el cliente. Esto es conveniente para aplicaciones compuestas por varios servlets.
- ✓ Si la sesión se ha diseñado para que persista más allá del cierre del navegador, en la cookie viajará el id de la sesión. Al volver al mismo sitio web, la cookie llevará el id previo y el servidor nos reconocerá con una sesión pendiente.



- Para utilizar el seguimiento de sesión debemos:
  - ✓ Obtener una sesión (un objeto HttpSession) para un usuario.
  - ✓ Almacenar u obtener datos desde el objeto HttpSession.
  - ✓ Invalidar la sesión (opcional).



## 3.1. Obtener una Sesión

- El método `getSession()` del objeto `HttpServletRequest` devuelve una sesión de usuario.
- La sintaxis es:  

```
HttpSession session = request.getSession(boolean create);
```
- Cuando llamamos al método con su argumento `create` como `true`, la implementación creará una sesión si no está previamente creada; si ya lo está, la localiza y la abre.
- Si el argumento es `false`, se indica que la página no puede tener sesiones.



## 3.2. Almacenar y obtener datos desde la Sesión

- El Interface HttpSession proporciona métodos para crear y recuperar sesiones.
- ✓ **setAttribute(java.lang.String name, java.lang.Object value)**
  - Establece un nombre de atributo y un valor para la sesión.
  - El nombre debe ser un String, y el valor cualquier objeto Java.
  - Ejemplo: `session.setAttribute("accessCount", accessCount);`



✓ **java.lang.Object getAttribute(java.lang.String name)**

- Devuelve el objeto especificado por su nombre como parámetro.

- Ejemplo:

```
Integer accesos =(Integer)session.getAttribute("accessCount");
```

✓ **boolean isNew( )**

- Indica si el cliente tiene una sesión previa abierta o no.

- Ejemplo:

```
if (session.isNew()){....}
```



### 3.3. Invalidar una Sesión

- Una sesión de usuario puede ser invalidada manual o automáticamente, dependiendo de donde se esté ejecutando el servlet. (Por ejemplo, el Java Web Server, invalida una sesión cuando no hay peticiones de página por un periodo de tiempo determinado).
- Invalidar una sesión **significa eliminar el objeto HttpSession y todos sus valores del sistema.**
- Para invalidar manualmente una sesión, se utiliza el método `invalidate()`, con el siguiente formato: `session.invalidate();`



## ○ Api de HttpSession

### Method Summary

|                       |                                                                                                                                                                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| java.lang.Object      | <a href="#"><u>getAttribute(java.lang.String name)</u></a><br>Returns the object bound with the specified name in this session, or null if no object is bound under the name.                                                                              |
| java.util.Enumeration | <a href="#"><u>getAttributeNames()</u></a><br>Returns an Enumeration of String objects containing the names of all the objects bound to this session.                                                                                                      |
| long                  | <a href="#"><u>getCreationTime()</u></a><br>Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.                                                                                                   |
| java.lang.String      | <a href="#"><u>getId()</u></a><br>Returns a string containing the unique identifier assigned to this session.                                                                                                                                              |
| long                  | <a href="#"><u>getLastAccessedTime()</u></a><br>Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT, and marked by the time the container recieved the request. |
| int                   | <a href="#"><u>getMaxInactiveInterval()</u></a><br>Returns the maximum time interval, in seconds, that the servlet container will keep this session open between client accesses.                                                                          |



|                                    |                                                                                                                                                                                                       |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">ServletContext</a>     | <a href="#">getServletContext()</a><br>Returns the ServletContext to which this session belongs.                                                                                                      |
| <a href="#">HttpSessionContext</a> | <a href="#">getSessionContext()</a><br><b>Deprecated.</b> <i>As of Version 2.1, this method is deprecated and has no replacement. It will be removed in a future version of the Java Servlet API.</i> |
| java.lang.Object                   | <a href="#">getValue(java.lang.String name)</a><br><b>Deprecated.</b> <i>As of Version 2.2, this method is replaced by <a href="#">getAttribute(java.lang.String)</a>.</i>                            |
| java.lang.String[]                 | <a href="#">getValueNames()</a><br><b>Deprecated.</b> <i>As of Version 2.2, this method is replaced by <a href="#">getAttributeNames()</a></i>                                                        |
| void                               | <a href="#">invalidate()</a><br>Invalidates this session then unbinds any objects bound to it.                                                                                                        |
| boolean                            | <a href="#">isNew()</a><br>Returns true if the client does not yet know about the session or if the client chooses not to join the session.                                                           |



void [putValue](#)(java.lang.String name, java.lang.Object value)

**Deprecated.** *As of Version 2.2, this method is replaced by*  
[setAttribute\(java.lang.String, java.lang.Object\)](#)

void [removeAttribute](#)(java.lang.String name)

Removes the object bound with the specified name from this session.

void [removeValue](#)(java.lang.String name)

**Deprecated.** *As of Version 2.2, this method is replaced by*  
[removeAttribute\(java.lang.String\)](#)

void [setAttribute](#)(java.lang.String name, java.lang.Object value)

Binds an object to this session, using the name specified.

void [setMaxInactiveInterval](#)(int interval)

Specifies the time, in seconds, between client requests before the servlet container will invalidate this session.



## ○ Ejemplo:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.net.*;
import java.util.*;

public class MostrarSession extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(true); //hay un sólo objeto por sesión
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String cabecera;
        Integer accessCount = new Integer(0);
        if (session.isNew()) {
            cabecera = "Bienvenido, Nuevo Usuario";
        }
        else {
            cabecera = "Bienvenido Usuario";
            Integer accesos =(Integer)session.getAttribute("accessCount");
            if (accesos != null) {
                accessCount =new Integer(accesos.intValue()+ 1);
            }
        }
    }
}
```



```
session.setAttribute("accessCount", accessCount); //se pueden añadir varios atributos al objeto
out.println("<HTML><BODY BGCOLOR='#FDF5E6'>\n" +
"<H1 ALIGN='CENTER'>" + cabecera + "</H1>\n" +
"<H2>Información de tu Session:</H2>\n" +
"<TABLE BORDER=1 ALIGN=CENTER>\n" +
"<TR BGCOLOR='#FFAD00'>\n" +
"<TH>Información<TH>Valor\n" +
"<TR>\n <TD>ID <TD>" + session.getId() +
"\n <TR>\n <TD>Fecha de Creación\n <TD>" +
new Date(session.getCreationTime()) + "\n" +
"<TR>\n<TD>Fecha de último acceso\n<TD>" +
new Date(session.getLastAccessedTime()) +
"\n<TR>\n" +
"<TD>Número de accesos previos \n <TD>" +
accessCount + "\n</TABLE>\n </BODY></HTML>");
}
```



## 3.4. Comunicación entre Servlets

- Utilizando el mecanismo de las sesiones podemos conseguir comunicar datos entre Servlets.
- Las sesiones son espacios de memoria comunes a todos los servlets que forman el sitio web.
- Si disponemos de un objeto Session llamado Datos, que sería un ArrayList de una clase Cuadro, podríamos establecer el siguiente ejemplo :

```
HttpSession Datos = request.getSession(true);
```

```
Datos.setAttribute("ArrayCuadros",ArrayCuadros);
```

y se puede utilizar en otro Servlet

```
ArrayCuadros=(ArrayList)Datos.getAttribute("ArrayCuadros");
```



### 3.5. Ceder el control desde un servlet a otro o una JSP

- Es tan sencillo como escribir:

```
response.sendRedirect("OfertaPartidosJSP.jsp");
```



## 3.6. Sesiones en JSP

- Existe un objeto implícito para crear sesiones llamado **session**, y se puede utilizar directamente, como si ya estuviese creado. Debemos recordar que los atributos de sesión deben ser objetos.
- Ejemplo de la aplicación “Acierta un Número” utilizando JSP.

El primer archivo sería una JSP en el que se muestra un campo de texto para introducir un número y el botón Enviar, que nos dirigirá a otra JSP llamada `jspAcierta.jsp`.

Este segundo archivo `jspAcierta.jsp` recogerá el numero enviado a través del formulario y utilizando dos atributos de sesión va dando pistas sobre el número a adivinar y el número de intentos que lleva el usuario que está jugando.



```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page session="false"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title>
  </head>
  <body>
    <p align="center"><font size="+1" color="#993300"><b><font size="+2">
      <i>Juego de Acierta un número</i></font></b></font></p>
    <br> <form action="jspAcierta.jsp">Dime un número :
    <input type="text" name="intento"
    size="5" maxlength="5">
    <br><br>
    <input type="submit" value="Enviar">
    </form>
  </body>
</html>
```



```
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>JSP Acierta</title>
</head>
<body>
<%@page import="javax.servlet.*"%>
<%@page import="javax.servlet.http.*"%>
<%@page import="java.io.PrintWriter"%>
<%@page import="java.io.IOException"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%!int prueba=0;%>
<% try
{
  prueba = Integer.parseInt(request.getParameter("intento"));
}
catch(Exception e)
{
  e.printStackTrace();
}%>
```



```
<%! int pensado=0;%>
<%! Integer pensa;%>
<%! Integer veces;%>
<%! int vecesJugador=0;%>
<% veces=new Integer(0);%>
<% if(session.isNew()) {
    pensado=(int)(Math.random()*100)+1;
    pensa=new Integer(pensado);
    session.setAttribute("numeroPC",pensa);
    session.setAttribute("vecesJuego",veces);
} else {
    pensa=(Integer)session.getAttribute("numeroPC");
    veces=(Integer)session.getAttribute("vecesJuego");
    pensado=pensa.intValue(); }
vecesJugador=veces.intValue();
vecesJugador++;
veces=new Integer(vecesJugador);
session.setAttribute("vecesJuego",veces); %>
```



```
<% if(prueba<pensado) {%>
  <p>El número pensado es mayor</p>
  <p align="center"><font size="+1" color="#993300"><b><font size="+2"><i>Juego de Acierta
un número</i></font></b></font></p>
  <br> <form action='jspAcierta.jsp'>Dime un número :
  <input type='text' name='intento'
size='5' maxlength='5'>
  <br><br>
  <input type='submit' value='Enviar'>
  </form>
<% }
else if(prueba>pensado) {%>
  <p>El número pensado es menor</p>
  <p align="center"><font size="+1" color="#993300"><b><font size="+2"><i>Juego de Acierta
un número</i></font></b></font></p>
  <br> <form action='jspAcierta.jsp'>Dime un número :
  <input type='text' name='intento'
size='5' maxlength='5'>
  <br><br>
  <input type='submit' value='Enviar'>
  </form>
```



```
<% }else {%>
  <p>Has acertado</p>
  <%session.invalidate();%>
  <A HREF='index.jsp'>Jugar de nuevo</A>
<% }%>
  <br>llevas <%= vecesJugador%> intentos
</body>
</html>
```